



Dossier Personnel BTS CIEL SESSION 2026 – IR E6 – PROJET TECHNIQUE

Ballon Stratosphérique



Lycée : Touchard-Washington		Ville : LE MANS	
Nom du projet :	Ballon Stratosphérique		
N° du projet :	TW4		
Projet nouveau	Oui ☒ Non ☐	Projet Interne	Oui ☒ Non ☐
Délai de réalisation	150 heures	Statut des étudiants	Formation initiale ☒ Apprentissage ☐
Spécialité des étudiants	ER ☐ IR ☒ Mixte ☐	Nombre d'étudiants	4 étudiants
Professeurs responsables	Anthony LE CREN, Jilali KHAMLACH		

Sommaire

Table des matières

1.Introduction.....	3
2.Cas d'utilisation.....	4
2.1.Récupérer les positions sur APSR.fi.....	4
2.2.Mettre à jour la base de donnée.....	5
2.3.Visualiser la position du véhicule par rapport au ballon.....	6
3.Choix technologiques.....	7
3.1.QT Creator, C++, WebSocket & JSON.....	7
3.2.Netbeans & JavaScript.....	8
3.3.MariaDB.....	9
3.4.GitHub.....	9
3.5 Structure du GitHub.....	10
4.Technologies Utilisées.....	11
4.1.OpenStreetMap.....	11
4.2.APSI et aprs.fi.....	12
.....	12
4.3.Fonctionnement d'une API.....	12
5.Diagrammes de séquences 5.1.Récupérer les positions sur APRS.fi & Mettre à jour la base de donnée.....	14
5.2.Visualiser la position du véhicule par rapport au ballon.....	16
6.Diagramme de classe.....	17
.....	18
7. Fiche de test.....	19
8.Diagramme de Gantt 9.Compétences acquises.....	21
10.Perspectives d'améliorations.....	22

1.Introduction

Dans le cadre de notre deuxième année de BTS CIEL option Informatique et Réseau, nous avons été amenés à réaliser un projet technique sur plusieurs mois. Ce projet nous a été proposé par nos professeurs encadrants, Anthony LE CREN et Jilali KHAMLACH.

Notre projet consiste à vérifier la loi des gaz parfait ($PV = nRT$) en se référant au volume du ballon stratosphérique. Afin de réaliser le projet dans les meilleures conditions, le CNES (Centre National d'Études Spatiales), agence spatiale et institut de recherche publique situé à Toulouse (département de la Haute-Garonne, 31) ainsi que l'association Planète Science espace, situé à Paris 8^e (département de Paris, 75) dans le Palais de la Découverte nous subventionne à hauteur de 2000€. Pour faciliter la récupération de la nacelle, le Radio Club de Sarthe est aussi impliqué dans le suivi du ballon-sonde. Durant ce projet, réalisé en équipe de quatre personnes, nous avons réparti les différentes tâches afin de pouvoir progresser plus efficacement et répondre au mieux au cahier des charges.

Ce projet est réalisé par: PAIN Iako, BRANDS Noé, KALO Harold et GUERRIAU Lucien.

2.Cas d'utilisation

2.1.Récupérer les positions sur APRS.fi

Récupérer les positions sur APRS.fi	
Pré-conditions :	L'unité centrale doit être connectée à Internet, avec un compte APSI.fi avec une clef API, puis envoyer la requête sur un navigateur ou par une méthode GET avec l'adresse ci-dessous et le dossier config.ini est configuré avec les bons paramètres afin d'avoir la bonne réponse: https://api.aprs.fi/api/get?name=INDICATIFRADIOAMATEUR&what=loc&apikey=APIKEY&format=json Les champs du config.ini sont : name ; what ; what_wx ; apikey ; format
Post-conditions :	L'API d'APRS.fi nous renvoie une réponse en format json et confirme le succès de la requête par les deux premières lignes de la réponse <pre>"command": "get", "result": "ok"</pre>
Scénario Principal :	Le programme QT6 effectue une requête à l'API d'APRS.fi toute les minutes. Si aucune erreur est détecté, les informations sont envoyées dans la base de données.
Alternative :	Les informations du config.ini sont erronées, l'API d'APRS.fi nous retourne une réponse avec le code de l'erreur: <pre>"command": "get", "result": "fail" "code": "apikey-invalid", "description": "invalid API key"</pre>

2.2.Mettre à jour la base de donnée

Mettre à jour la base de donnée	
Pré-conditions :	L'unité centrale servant de serveur d'hébergement pour la base de donnée est alimenté et allumé. Le dossier config.ini est configuré avec les bons paramètres afin d'avoir accès à la base de donnée. Les champs sont : host ; username ; password ; database
Post-conditions :	Le programme nous informe de la mise en base de donnée via les logs : Sauvegarde BDD OK pour : F4KMN-8
Scénario Principal :	Après avoir récupérer la réponse de l'API d'APRS.fi, le programme incorpore les données dans trois tables : <u>HISTORIQUE</u> : Inclut toutes les requêtes « loc » en prenant en compte tout les champs. <u>POSITIONS</u> : Inclut seulement les DERNIÈRES positions (deux, celle du ballon et celle de la voiture) en prenant en compte seulement les champs : LAT, LNG, NAME, LASTTIME <u>TELEMETRIES</u> : Inclut toutes les requêtes « wx » en prenant en compte tout les champs.
Alternatives :	Les informations du config.ini sont erronées, les logs du programmes nous indique l'erreur.

2.3. Visualiser la position du véhicule par rapport au ballon

Visualiser la position du véhicule par rapport au ballon	
Pré-conditions :	L'unité centrale servant de serveur d'hébergement pour le site est alimenté et allumé. Le programme QT6 est en marche.
Post-conditions :	L'utilisateur peut avoir le site internet sur son périphérique.
Scénario Principal :	Une fois la requête API d'APRS.fi effectuée, la WebSocket prend les positions du ballon et de la voiture suiveuse dans la base de donnée via la table POSITIONS. Ces dernières sont ensuite envoyées à tout les clients connectés. Dès lors les positions seront affichés sur l'interface utilisateur du site (Une carte OSM) avec les informations concernant la position (latitude et longitude) ainsi que la télémétrie (température, pression, humidité)
Alternatives :	Le programme QT6 n'est pas en marche. De ce fait, la WebSocket n'est point en route, le client ne reçoit aucune information. Si c'est l'unité centrale qui n'est pas en marche, alors le client n'aura pas accès à l'interface. Ou alors le client n'a pas d'accès à internet, dès lors la connexion est impossible.

3.Choix technologiques

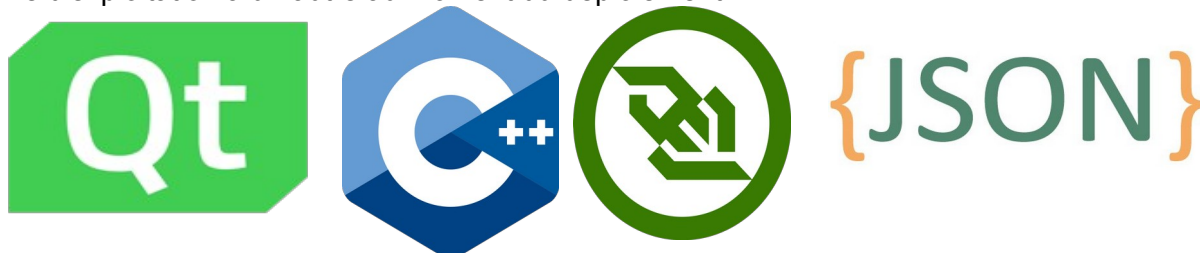
3.1.QT Creator, C++, WebSocket & JSON

J'ai choisi d'utiliser le framework Qt pour développer mon serveur backend en C++, car il me permet de transformer un langage natif de bas niveau en une véritable boîte à outils réseau prête à l'emploi. Le C++ standard étant historiquement dépourvu de bibliothèques natives dédiées à la communication réseau ou à la manipulation de données web, un développement from scratch m'aurait contraint à assembler manuellement de multiples dépendances externes complexes — ce qui aurait considérablement alourdi la configuration et la maintenance du projet.

Qt résout ce problème en offrant un écosystème clés en main, totalement unifié. Concrètement, j'ai utilisé QNetworkAccessManager dans la classe ApiClient pour envoyer des requêtes HTTP GET périodiques vers l'API aprs.fi — deux instances distinctes fonctionnant en parallèle, l'une pour les positions (loc) et l'autre pour les télémétries météo (wx). Les réponses JSON sont ensuite traitées via QJsonDocument et QJsonArray avant d'être transmises au DatabaseManager pour persistance en base MariaDB, puis diffusées aux clients web connectés par le WebSocketServer via QWebSocketServer. Ces modules s'intègrent nativement et communiquent de façon fluide via les mêmes structures de données, ce qui rend le code à la fois cohérent et lisible.

La nature de mon application impose par ailleurs une gestion rigoureuse de la concurrence : le serveur doit simultanément interroger l'API aprs.fi selon un intervalle configurable, analyser les réponses JSON, effectuer des opérations SQL sur trois tables distinctes — HISTORIQUE, POSITIONS et TELEMETRIES — et diffuser en temps réel les mises à jour à tous les clients web connectés. C'est là que l'architecture orientée événements de Qt, fondée sur son système de signaux et de slots, prend tout son sens. Grâce à sa boucle d'événements très optimisée, l'ensemble de ces tâches est rendu totalement asynchrone : lorsque l'ApiClient émet le signal dataReceived, la fenêtre principale le reçoit via un slot, déclenche la sauvegarde en base, puis appelle immédiatement broadcastPositions sur le WebSocketServer — le tout sans jamais bloquer le programme dans l'attente d'une réponse réseau, et sans avoir à gérer manuellement des threads complexes.

Enfin, ce choix architectural renforce la stabilité du cycle de vie de l'application. La manipulation du JSON est prise en charge de manière sûre par les classes dédiées de Qt, ce qui élimine les risques de plantage liés à un parsing défaillant. Je tire également parti du modèle d'arborescence des objets (QObject) et de la méthode deleteLater() pour le nettoyage asynchrone des sockets clients lors de leur déconnexion — une protection efficace contre les fuites mémoire, qui représentent habituellement un défi sérieux lors de la manipulation de sockets réseau en C++. Au final, cette approche me garantit un serveur performant, stable, et facilement portable d'un système d'exploitation à un autre au moment du déploiement.



3.2. Netbeans & JavaScript

Pour la partie frontale, destinée à l'utilisateur final, le choix de JavaScript s'est imposé naturellement. L'objectif était de proposer une interface accessible depuis n'importe quel navigateur web — que ce soit sur ordinateur ou sur mobile — sans aucune installation préalable côté client. Dans ce contexte, JavaScript constitue la seule option véritablement universelle.

C'est sur la gestion du temps réel que ce langage a démontré toute sa pertinence pour ce projet. Le serveur Qt/C++ interroge périodiquement l'API `aprs.fi` pour récupérer les positions et les télémétries des indicatifs suivis, les stocke en base de données MariaDB, puis diffuse les données mises à jour à tous les clients connectés via un serveur WebSocket. Côté navigateur, JavaScript prend en charge les WebSockets de manière native et particulièrement efficace. Dès réception d'un message de type `position_update`, le code JavaScript intercepte la donnée JSON — contenant le nom de la station, ses coordonnées GPS, sa température, son humidité et sa pression atmosphérique — et met à jour l'interface instantanément sans aucun rechargement de page. Les marqueurs correspondant aux indicatifs suivis sont alors positionnés ou actualisés sur la carte OpenStreetMap, et un clic sur l'un d'eux affiche une fenêtre récapitulant l'ensemble de ces informations. Ce mécanisme a également facilité l'intégration de bibliothèques tierces, notamment Leaflet pour l'affichage de la carte interactive.

Pour le développement de cette partie, j'ai opté pour l'IDE NetBeans. On pourrait légitimement s'interroger sur la pertinence d'un environnement aussi complet pour du JavaScript, un langage qui peut techniquement s'écrire dans un simple éditeur de texte. En pratique, dès lors que le projet implique plusieurs fichiers HTML, des feuilles de style CSS et de multiples scripts gérant à la fois la connexion WebSocket, le traitement des messages JSON et l'interaction avec la carte, la lisibilité et la maintenabilité du code deviennent des enjeux réels.

NetBeans répond à cette problématique en centralisant l'ensemble du projet au sein d'un environnement unique et structuré. Il offre une complétion de code avancée, une détection syntaxique en temps réel, et intègre des outils permettant de tester l'interface directement en environnement local, sans recourir à des outils externes supplémentaires. Cette centralisation a sensiblement réduit la friction liée à la gestion multi-fichiers et contribué à maintenir une base de code cohérente tout au long du développement.

Au final, l'association de NetBeans et de JavaScript a permis de concevoir une interface dynamique et réactive, communiquant de façon fiable et fluide avec le serveur C++/Qt via WebSocket.



3.3.MariaDB



Pour la persistance des données, j'ai retenu MariaDB comme système de gestion de base de données. Face au volume continu de positions GPS et de relevés météorologiques en provenance de l'API, il était indispensable de disposer d'un moteur robuste, capable d'absorber un flux soutenu de requêtes en écriture sans dégradation des performances. Le modèle relationnel de MariaDB s'est révélé particulièrement adapté pour structurer de manière claire et cohérente les différentes tables du projet — historique des positions, position courante et données de télémétrie.

L'argument décisif a toutefois été sa compatibilité native avec l'environnement Qt. MariaDB étant pleinement compatible avec le protocole MySQL, son intégration au sein du serveur C++ s'est effectuée de manière transparente, avec un minimum de configuration.

Au final, le choix de MariaDB repose sur un équilibre entre fiabilité éprouvée, simplicité d'intégration et adéquation directe aux contraintes techniques du projet.

3.4.GitHub



Tout au long du développement, j'ai utilisé Git comme système de contrôle de version, avec GitHub comme plateforme d'hébergement du dépôt. Au-delà de la simple sauvegarde du code source, cette approche m'a permis de maintenir un historique précis et structuré de l'ensemble des modifications apportées au projet. Chaque fonctionnalité ou correction a fait l'objet d'un commit distinct, accompagné d'un message explicite, offrant ainsi une traçabilité complète de l'évolution du code. Cette discipline de versionnement s'est révélée particulièrement précieuse lors des phases de débogage, en permettant d'identifier rapidement l'origine d'une régression et de revenir à un état stable si nécessaire. GitHub a également sécurisé le projet contre tout risque de perte de données, tout en constituant une base solide pour une éventuelle collaboration ou une reprise ultérieure du projet.

3.5 Structure du GitHub

main 5 Branches 0 Tags Go to file T Add file <> Code

HaroldKalo Update index.php 9368e10 · last week 56 Commits

Documentation	Delete Documentation/Dossier_de_conception_remarque...	2 weeks ago
Nacelle	Creation du fichier nacelle	last month
Sol	Update index.php	last week
README.md	Update Noé's role description in README	2 months ago

lucien 5 Branches 0 Tags Go to file T Add file <> Code

This branch is 26 commits ahead of and 55 commits behind main . #2

Iguerriau Rajout doc Doxygen 8af415e · 2 weeks ago 27 Commits

Projet Fini	Rajout doc Doxygen	2 weeks ago
Versionnage	Versionnage des fichiers	2 weeks ago
Dossier d'analyse & Conception TW4-PBS.pdf	Add files via upload	3 months ago
test	OK	3 months ago

ballonstratospherique / Projet Fini Add file ...

Iguerriau Rajout doc Doxygen 8af415e · 2 weeks ago History

This branch is 26 commits ahead of and 55 commits behind main . #2

Name	Last commit message	Last commit date
..		
OSMPBS	Versionnage des fichiers	2 weeks ago
RecupPositionsAPRS	Rajout doc Doxygen	2 weeks ago
TestUnitaireDataBaseManager	Versionnage des fichiers	2 weeks ago
aprs_send_test.py	Versionnage des fichiers	2 weeks ago
ballon2026v2.sql	Mise au propre + maj BDD	2 weeks ago

4. Technologies Utilisées

4.1. OpenStreetMap

OpenStreetMap (OSM) est une base de données géographique mondiale, libre et collaborative, lancée en 2004 par Steve Coast. Son principe repose sur la contribution communautaire : n'importe qui peut ajouter, modifier ou corriger des données géographiques, qui sont ensuite disponibles gratuitement sous licence ODbL (Open Database Licence).

La base de données recense des milliards d'éléments — routes, bâtiments, cours d'eau, points d'intérêt — collectés via des relevés GPS, des images satellites et des connaissances locales. Ces données sont accessibles par une API REST et peuvent être intégrées dans des applications via des bibliothèques comme Leaflet ou OpenLayers. Des outils complémentaires permettent également le géocodage (Nominatim), le calcul d'itinéraires (OSRM) ou l'interrogation avancée de la base (Overpass API).

OSM est aujourd'hui utilisé dans de nombreux secteurs : gestion de crise et aide humanitaire, mobilité, logistique, aménagement du territoire et systèmes d'information géographique (SIG). Sa principale force est la réactivité de sa mise à jour, assurée par une communauté mondiale de contributeurs. En contrepartie, la qualité des données peut varier selon les régions, ce qui nécessite parfois une vérification préalable à leur utilisation dans un contexte professionnel.

Dans le cadre de ce projet, OSM a été retenu pour sa gratuité, sa flexibilité d'intégration et la richesse de sa couverture géographique.



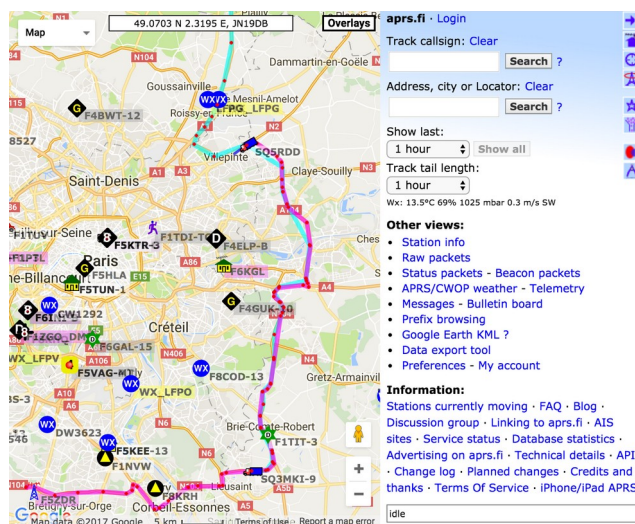
OpenStreetMap France

4.2.APSI et aprs.fi

APRS (Automatic Packet Reporting System) est un protocole de communication numérique utilisé en radio amateur, développé par Bob Bruninga dans les années 1980. Il permet la transmission en temps réel de données de positionnement GPS, de messages texte et d'informations télémétriques via des fréquences radio dédiées. Chaque station APRS émet périodiquement des trames contenant sa position et diverses informations, qui sont ensuite relayées par un réseau de répéteurs (digipeaters) et de passerelles internet (iGates).

aprs.fi est une plateforme web qui agrège et visualise l'ensemble de ces données sur une carte interactive. Elle expose une API publique permettant d'interroger en temps réel la base de données des stations APRS. Les requêtes s'effectuent via des appels HTTP avec une clé d'authentification, et les réponses sont retournées au format JSON.

Dans le cadre de ce projet, l'API aprs.fi est interrogée pour récupérer, pour chaque indicatif suivi, sa position géographique ainsi que ses relevés télémétriques : température, humidité et pression atmosphérique. Ces données sont ensuite stockées dans une base de données MariaDB, puis restituées sur une application web intégrant une carte OpenStreetMap. Chaque indicatif y est représenté par un point positionné selon ses coordonnées GPS. Un clic sur ce point ouvre une fenêtre affichant les informations associées à la station : sa localisation précise ainsi que ses dernières mesures environnementales.



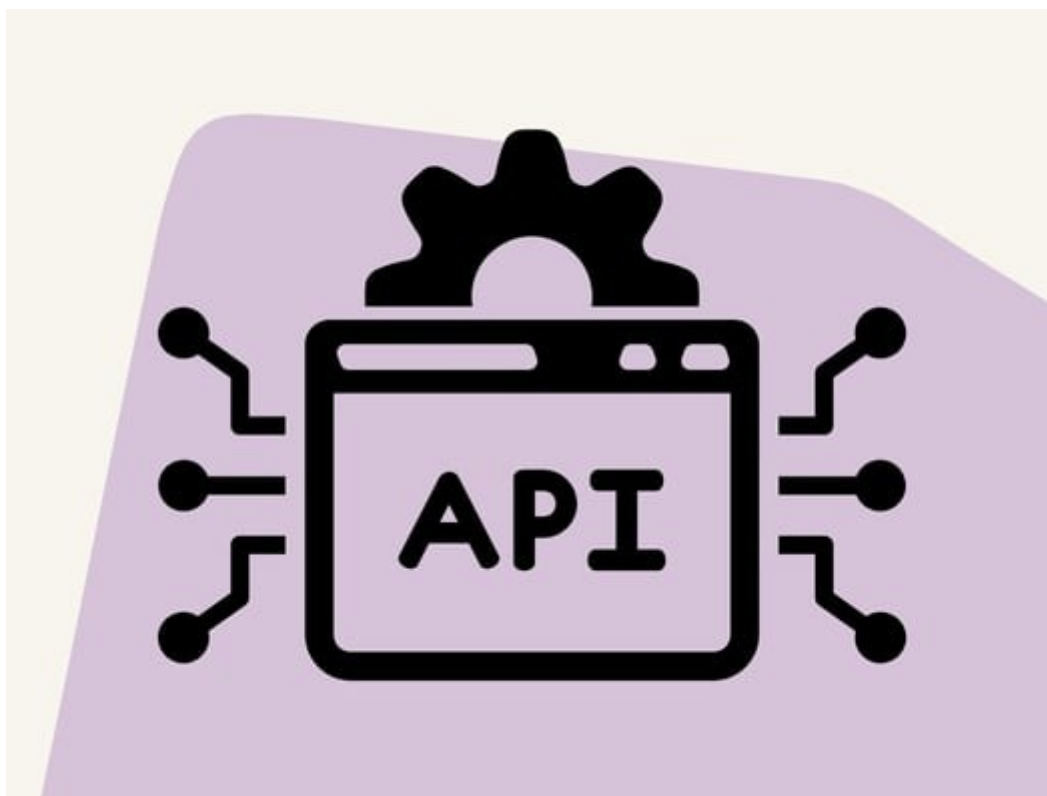
4.3.Fonctionnement d'une API

Une API (Application Programming Interface) est une interface logicielle qui permet à deux applications de communiquer entre elles de manière standardisée. Elle définit un ensemble de règles et de points d'accès, appelés endpoints, par lesquels une application peut envoyer des requêtes et recevoir des données en retour, sans avoir accès directement à la logique interne du service interrogé.

Dans le cadre du web, les API dites REST (Representational State Transfer) sont les plus répandues. Elles reposent sur le protocole HTTP et utilisent des méthodes standardisées pour interagir avec les données : GET pour lire, POST pour créer, PUT pour modifier, ou DELETE pour supprimer. Chaque requête est adressée à une URL spécifique, éventuellement accompagnée de paramètres, et le serveur retourne une réponse généralement formatée en JSON, un format texte léger et facilement exploitable par les applications.

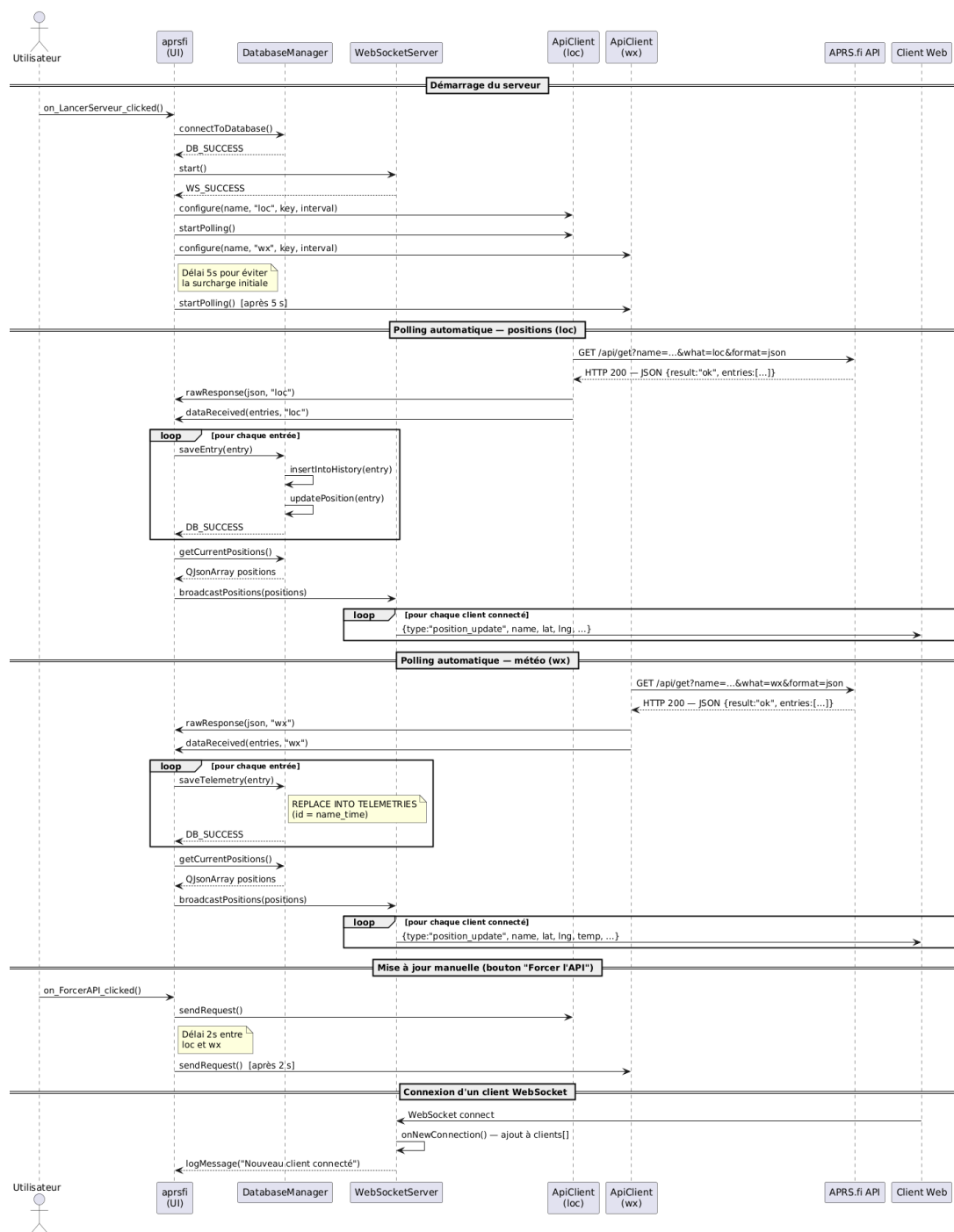
L'accès à une API est souvent sécurisé par une clé d'authentification, transmise dans la requête pour identifier l'appelant et contrôler les droits d'accès.

Dans ce projet, c'est ce mécanisme qui est utilisé pour interroger l'API d'aprs.fi. Une requête HTTP GET est envoyée vers l'endpoint correspondant, en précisant les indicatifs souhaités ainsi que la clé d'authentification. Le serveur retourne alors une réponse JSON contenant les données de position et les relevés télémétriques (température, humidité, pression) de chaque station. Ces données sont ensuite extraites et insérées dans la base de données MariaDB pour être exploitées par l'application web.



5. Diagrammes de séquences

5.1. Récupérer les positions sur APRS.fi & Mettre à jour la base de donnée



Description :

Ce diagramme de séquence décrit ce qui se passe côté serveur, derrière l'interface graphique que l'utilisateur voit sur son écran.

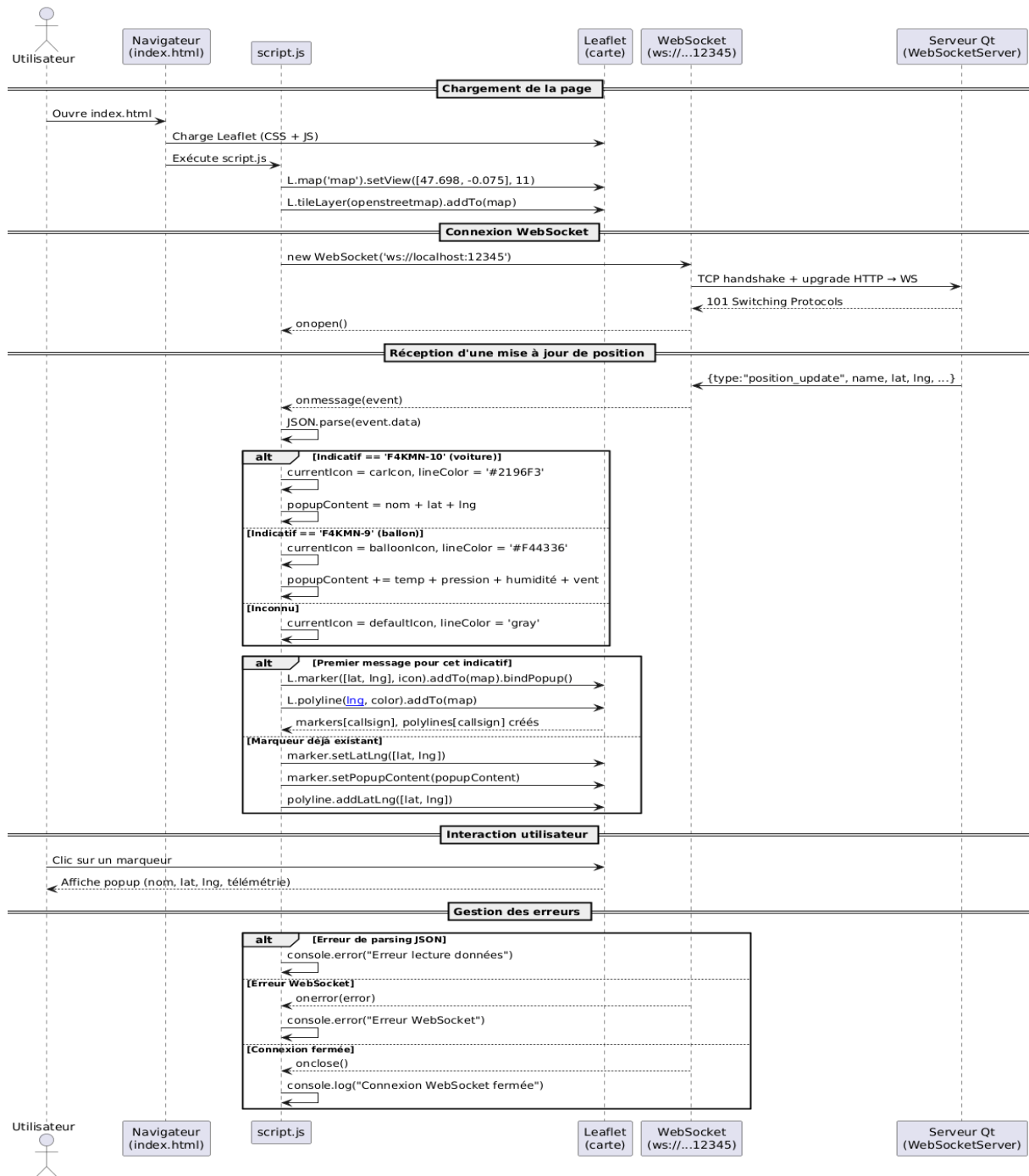
Au démarrage — déclenché par un clic sur le bouton dédié — le serveur s'initialise étape par étape : connexion à la base de données, lancement du WebSocket, puis configuration des deux clients API chargés de récupérer les données sur APRS.fi. Le client météo démarre volontairement avec 5 secondes de décalage par rapport au client de localisation, histoire de ne pas tout envoyer d'un coup sur le réseau.

Une fois lancés, ces deux clients tournent en tâche de fond et interrogent l'API à intervalles réguliers. À chaque réponse, les données JSON sont parsées, enregistrées dans la base (historique des positions d'un côté, télémétrie météo de l'autre), puis immédiatement rediffusées à tous les clients web connectés en WebSocket — c'est ce mécanisme qui assure la mise à jour en temps réel de la carte.

L'utilisateur garde aussi la main à tout moment : un bouton "Forcer l'API" déclenche une interrogation immédiate des deux sources, avec un léger décalage de 2 secondes entre les deux requêtes pour éviter les collisions.

Enfin, chaque nouvelle connexion d'un client web est détectée et enregistrée automatiquement, ce qui permet au serveur de maintenir à jour la liste des destinataires pour ses diffusions

5.2. Visualiser la position du véhicule par rapport au ballon



Description :

Ce diagramme de séquence montre comment l'interface web fonctionne de bout en bout, du premier chargement jusqu'à l'affichage des données en direct.

Tout commence par l'ouverture de la page : le navigateur charge la carte interactive grâce à Leaflet, puis ouvre une connexion WebSocket persistante avec le serveur Qt — c'est ce canal qui permet de recevoir les données en continu, sans avoir à rafraîchir la page.

Dès qu'un message arrive du serveur, le script l'analyse et adapte l'affichage selon la nature de l'émetteur : un marqueur bleu et une icône voiture pour les balises embarquées dans les véhicules, un marqueur rouge avec les données météo (température, pression, humidité, vent) pour les ballons, et une icône neutre pour tout indicatif non reconnu. Chaque nouvel émetteur génère automatiquement un marqueur et un tracé de trajet sur la carte ; pour les suivants, seule la position est mise à jour.

L'utilisateur peut à tout moment cliquer sur un marqueur pour consulter les dernières informations disponibles sous forme de popup.

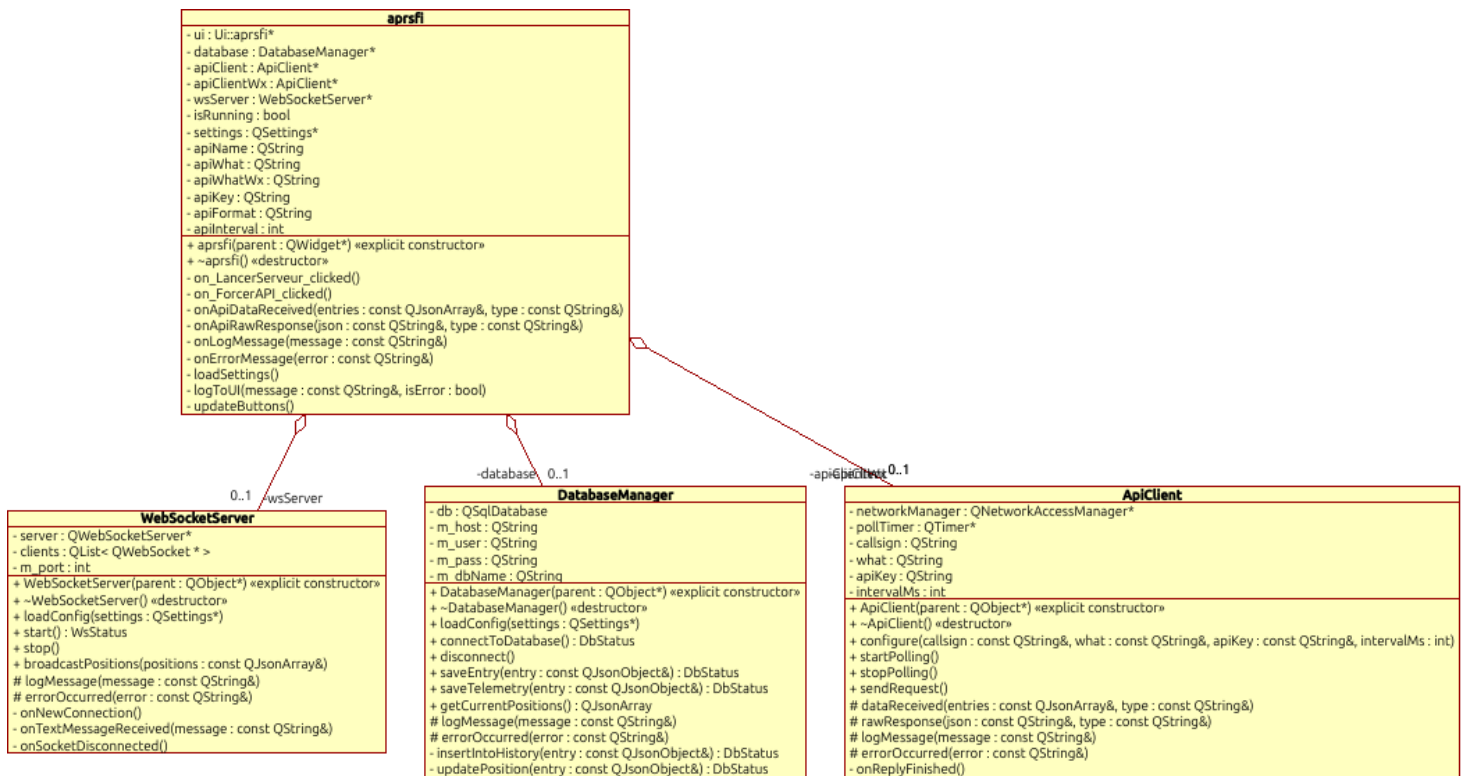
En cas de problème — erreur de parsing JSON, coupure WebSocket ou fermeture de connexion — le système le détecte et le consigne silencieusement dans la console, sans interrompre le reste du fonctionnement.

6. Diagramme de classe

Puisque nous avons effectué le choix des logiciels, nous pouvons désormais parfaire nos diagrammes afin de les adapter à nos outils. Dans ce diagramme de classe, nous avons donc intégré les classes fournies par le framework Qt (utilisé via QtCreator) :

- `QNetworkAccessManager` : permet d'effectuer des requêtes HTTP (réseau) afin d'interroger l'API distante (APRS.fi) et d'en récupérer les données.
- `QJsonObject` / `QJsonArray` : permettent la manipulation, la lecture et l'écriture de données structurées au format JSON (indispensable pour traiter les réponses de l'API et formater les trames à envoyer aux clients).
- `QSqlDatabase` : permet de gérer la connexion à la base de données métier et d'y exécuter des requêtes (pour sauvegarder les positions, l'historique et la télémétrie).
- `QWebSocket` / `QWebSocketServer` : permettent d'instancier un serveur et de gérer des échanges de données en temps réel. De plus, la classe `QWebSocket` est combinée à la classe `QList`, ce qui nous permet de créer et maintenir une liste des clients actuellement connectés (clients : `QList<QWebSocket*>`).
- `QTimer` : permet la création d'un minuteur (ici `pollTimer`) nous servant à déclencher des requêtes régulières vers l'API selon un intervalle défini.

- QSettings : permet de lire les données présentes dans un fichier de configuration afin de charger les paramètres nécessaires au démarrage (identifiants de la BDD, clés API, paramètres du serveur).
- QWidget / QObject : classes de base de l'environnement Qt permettant respectivement de générer l'interface graphique (pour la classe principale aprsfi) et de gérer la communication inter-objets via les signaux et slots.



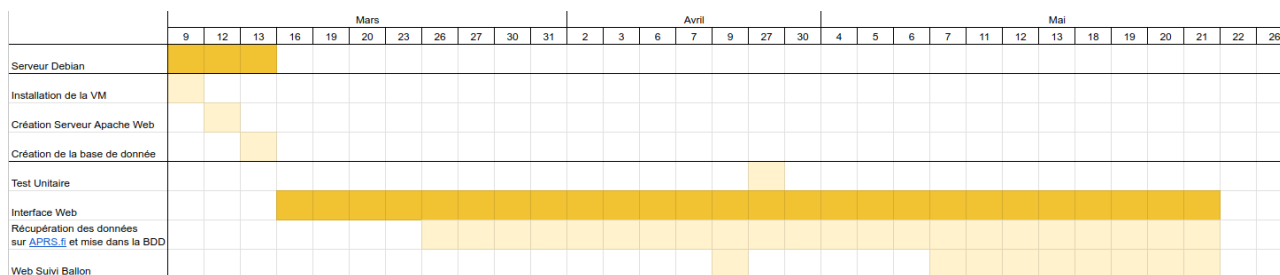
7. Fiche de test

Cette fiche de test permet de vérifier le stockage des données au sein de la BDD. La série de tests effectuée permettra de prendre en compte toutes les éventualités. Lors de l'élaboration de cette fiche de test seulement certaines classes étaient créées, une interface graphique a dû être créée afin de pouvoir réaliser les tests.

Référence du module testé	F01.DB.01
Date du test	04/06/2026
Type du test	Test fonctionnel et d'intégration
Objectif du test	Vérifier l'exactitude du stockage des données (positions APRS et télémetries météo) au sein de la base de données MariaDB, le respect des contraintes d'unicité (mise à jour vs insertion) et le traitement des erreurs de connexion.
Conditions du test	
État initial :	Le service MariaDB est actif et en cours d'exécution. • Les tables HISTORIQUE, POSITIONS et TELEMETRIES sont préalablement vidées pour éviter les faux positifs lors des tests d'insertion. • Le fichier config.ini est présent et configuré avec les bons identifiants de la base de données MariaDB.
Environnement du test :	Système d'exploitation : Linux Debian. • Interface graphique de test : Application Qt (aprsfi) lancée manuellement, comprenant la zone de "LogsProgramme" et les boutons "Lancer le serveur" et "Forcer l'API".
Procédures du test	
Opérations (Actions menées sur l'IHM ou le	Résultats attendus (Comportement du système

système)	et BDD)
Connexion valide : Démarrer l'application avec un config.ini valide et cliquer sur "Lancer le serveur".	L'interface affiche le log [INFO] Connexion réussie a la BDD.... La connexion MariaDB est établie et l'application lance les requêtes API (LOC et WX).
Erreur de connexion : Démarrer l'application avec un mot de passe ou un utilisateur erroné dans config.ini, puis cliquer sur "Lancer le serveur".	L'interface affiche [ERREUR] Erreur de connexion BDD... (avec le retour d'erreur natif MariaDB) puis [ERREUR] ERREUR FATALE.... Les appels à l'API et au WebSocket ne se lancent pas.
Insertion d'une nouvelle position (LOC) : Cliquer sur "Forcer l'API" pour réceptionner une trame d'une station qui n'est pas encore dans la base.	L'application exécute un INSERT dans la table HISTORIQUE et un INSERT dans la table POSITIONS. Le log affiche [INFO] Sauvegarde BDD OK pour : [nom de la station].
Mise à jour d'une position existante (LOC) : Réception d'une nouvelle trame de position pour une station déjà existante dans la base.	L'application exécute un INSERT dans HISTORIQUE mais effectue un UPDATE de la table POSITIONS. Le nombre total de lignes dans POSITIONS ne doit pas augmenter pour cette station précise.
Insertion d'une télémétrie inédite (WX) : Réception d'une trame de télémétrie météo inédite.	L'application effectue un REPLACE INTO dans la table TELEMETRIES en générant l'identifiant unique name_time. Le log affiche [INFO] Sauvegarde Telemetrie OK pour : [nom de la station].
Remplacement d'une télémétrie en doublon (WX) : Réception d'une trame météo ayant exactement le même nom et le même horodatage (time) qu'une trame existante.	Le moteur MariaDB exécute le REPLACE INTO de façon silencieuse : l'ancienne ligne est écrasée sans erreur SQL. Le système reste stable et ne lève pas d'exception de violation de clé primaire.
Récupération des données fusionnées : Vérifier le bon formatage des données via la méthode getCurrentPositions().	La requête de jointure (LEFT JOIN) s'exécute correctement sur MariaDB. La méthode retourne un tableau JSON contenant les positions jointes avec la <i>dernière</i> entrée météo connue (grâce à MAX(time)).

8. Diagramme de Gantt



9. Compétences acquises

La réalisation de ce projet de visualisation cartographique des stations APRS a permis d'acquérir et d'approfondir un large panel de compétences techniques et professionnelles, en phase avec les réalités du milieu des systèmes numériques et des réseaux.

En matière technique, le projet a permis :

- De concevoir et développer une application backend en C++ avec le framework Qt, en exploitant notamment QNetworkAccessManager pour les appels HTTP, QWebSocketServer pour la diffusion temps réel, et QJsonDocument pour le traitement des données.
- De mettre en œuvre une communication réseau complète, depuis l'interrogation d'une API REST externe (aprs.fi) jusqu'à la diffusion des données aux clients web via WebSocket.
- D'implémenter et de gérer une base de données relationnelle MariaDB, structurée autour de trois tables (HISTORIQUE, POSITIONS, TELEMETRIES), avec des requêtes SQL paramétrées via QSqlDatabase et QSqlQuery.
- De concevoir une base de données normalisée intégrant un mécanisme d'historisation des positions et des relevés télémétriques (température, humidité, pression).
- De développer une interface web dynamique en JavaScript, exploitant les WebSockets pour la mise à jour en temps réel des marqueurs sur une carte OpenStreetMap via la bibliothèque Leaflet.
- De manipuler des formats d'échange de données JSON à chaque niveau de la chaîne de traitement, de la réponse API jusqu'au message diffusé au navigateur.
- De mettre en place une architecture logicielle modulaire et orientée événements, fondée sur le système de signaux et slots de Qt, garantissant l'asynchronisme des opérations sans gestion manuelle de threads.

D'autre part, au niveau professionnel, ce projet a également permis :

- D'appliquer une démarche de conception rigoureuse, depuis l'analyse du besoin jusqu'au choix des technologies et à leur intégration cohérente.
- De lire et d'exploiter une documentation technique en anglais (API aprs.fi, protocole APRS, bibliothèques Qt).
- D'entretenir un esprit d'analyse, d'autonomie et de résolution de problèmes concrets, notamment face aux contraintes de synchronisation des requêtes et de cohérence des données.
- De documenter correctement les étapes du projet, les choix techniques et les résultats obtenus, à destination d'un public technique.

Pour résumer, ce projet constitue l'illustration d'une expérience complète ayant permis de mobiliser les compétences du référentiel BTS CIEL IR et de se confronter à des situations techniques proches du réel : intégration de services web, persistance des données, communication réseau et restitution visuelle de l'information.

10.Perspectives d'améliorations

La version actuelle du projet constitue une base fonctionnelle solide, mais plusieurs axes d'évolution pourraient être envisagés pour enrichir l'application et la rapprocher davantage d'une solution professionnelle.

Sur le plan technique, il serait envisageable de :

- **Ajouter un historique visuel sur la carte** : afficher la trajectoire d'un indicatif sous forme de tracé sur la carte OpenStreetMap, en exploitant les données déjà présentes dans la table HISTORIQUE (champs lat, lng, time).
- **Mettre en place des graphiques de télémétrie** : représenter l'évolution de la température, de l'humidité et de la pression dans le temps via des graphiques dynamiques côté web, à partir des données stockées dans la table TELEMETRIES.
- **Implémenter un système d'alertes** : déclencher une notification visuelle ou sonore lorsqu'une valeur télémétrique dépasse un seuil défini (par exemple, température trop élevée), directement dans l'interface web.
- **Sécuriser la communication WebSocket** : passer en mode wss:// (WebSocket sécurisé via TLS) pour chiffrer les échanges entre le serveur Qt et le navigateur, ce qui est indispensable dans un contexte de déploiement réel.
- **Enrichir la base de données** : ajouter une table dédiée aux alertes ou aux seuils configurables par indicatif, afin de rendre le système plus paramétrable sans modification du code.

- **Ajouter une authentification** : protéger l'interface web par un système de login simple, pour restreindre l'accès aux données des stations.

Sur le plan de l'architecture, une évolution naturelle serait de déployer le serveur Qt sur un système embarqué de type Raspberry Pi, ce qui permettrait de disposer d'un serveur autonome, peu consommateur d'énergie et fonctionnant en continu.